ShowUp

Complete Project Documentation

Large-Scale Event Organizing Platform

Backend: Python / Flask REST API
Frontend: Kotlin / Jetpack Compose Android Application
Maps, anonymous authentication, moderation, and security architecture

---

Document type:	Examination-ready technical documentation
Prepared:	May 2026
Primary brand:	ShowUp
Accent color:	Atomic Orange (#FF4D00)

This edition reframes the project as a generalized event coordination platform for parades, marathons, civic demonstrations, community gatherings, and other high-participation public events.

Contents

1 Document Scope and Naming Clarification

This document presents **ShowUp** as a privacy-first, mobile-first platform for organizing and discovering large-scale public events. The system supports community gatherings, parades, marathons, civic demonstrations, public campaigns, and other coordinated activities where location, timing, updates, and participant interaction matter.

The project has undergone a product-direction overhaul. As a result, some implementation artifacts still contain earlier protest-related names, including route namespaces such as `/protests`, database files such as `protests.db`, Android classes such as `ProtestTheme.kt`, and variables such as `RATELIMIT_PROTESTS_PER_DAY`. These should be read as legacy technical identifiers for the broader **event-management domain**, not as a limitation of the current product scope. Product-facing language uses **events**, **public gatherings**, and **large-scale coordination** unless an exact code identifier or endpoint must be preserved for technical accuracy.

Examination positioning
ShowUp should be described as a large-scale event organization platform, not as a single-purpose application. Its distinguishing value is the combination of map-based discovery, anonymous account creation, reputation-aware moderation, and strong anti-abuse controls. Legacy endpoint names are implementation details created before the product was broadened.

2 Project Overview and Purpose

2.1 Project Identity

Item	Description
Project name	ShowUp
Backend repository	`protestbackend` (legacy repository name)
Android repository	`new-showup-main`
Primary platform	Native Android application with a Flask REST backend
Product category	Privacy-first event discovery, creation, and coordination
Core use cases	Parades, marathons, public gatherings, civic demonstrations, social campaigns, meetups, and other location-based events

2.2 Purpose

ShowUp is a privacy-first social coordination platform for large-scale events. It is built around fast event creation, geospatial discovery, anonymous participation, and moderated community interaction. Users can create events, publish updates, browse a mixed feed, view event locations on a map, join or endorse events, and interact through posts and comments.

The system consists of two major components:

- **Backend REST API**: A Python/Flask service that handles anonymous accounts, event resources, posts, comments, AI-assisted moderation, reputation tracking, geospatial search, rate limiting, security middleware, and human moderator workflows.
- **Android frontend**: A Kotlin/Jetpack Compose application named ShowUp, featuring a Mapbox-powered map, anonymous authentication, a morphing bottom-sheet interface, a side navigation drawer, event and post detail screens, user-specific lists, and a custom visual identity built around the atomic orange accent color.

The account model intentionally avoids email, phone numbers, and social login. Users create an account with a nickname and receive a secret account number used for authentication. This design lowers onboarding friction and reduces the amount of personal information stored by the platform.

2.3 Core Capabilities

Capability	Explanation
Anonymous accounts	Users authenticate with a generated secret account number rather than email, phone number, or social identity.
Geospatial event discovery	Events and posts can be searched by center point, radius, or bounding box. The platform enforces geographic limits to keep queries practical.
Event lifecycle actions	Users can create, update, delete, join, leave, endorse, and unendorse event resources.
Community feed	Events and posts are merged into a single feed, with an intentional event-to-post ratio to keep the experience event-centered.
AI-assisted moderation	Content is filtered through keyword checks and AI analysis, then either accepted, rejected, or sent to human review.
Human review workflow	Moderators can claim queue items, review ambiguous submissions, and manage approval or denial decisions atomically.
Security controls	The backend includes user-agent filtering, honeypots, proof-of-work, rate limiting, HMAC request signing support, image validation, and ban systems.
Brand system	ShowUp uses a dark-first visual language, Space Grotesk/Space Mono typography, and atomic orange as the primary accent.

3 System Architecture Overview

3.1 Architectural Framing

ShowUp follows a mobile-client, REST-backend architecture. The Android application owns presentation, map interaction, local session persistence, and user workflows. The Flask backend owns authentication, content persistence, moderation, geospatial queries, rate limiting, and security enforcement.

Because the project was redirected from a narrower civic-demonstration concept into a general large-scale event platform, several routes and files still use protest-related names. In architectural terms, these names represent the **event resource module**. For example, `/protests/create` should be understood as the current event-creation endpoint until a future `/events` alias or migration layer is introduced.

3.2 High-Level System Diagram

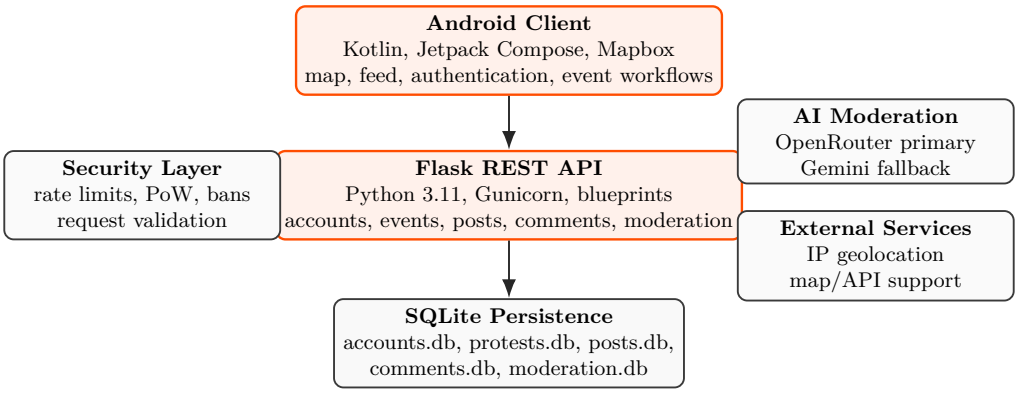

Figure 1: High-level ShowUp architecture. The Android client communicates with a Flask REST API, which coordinates persistence, moderation, geospatial behavior, and security enforcement.

3.3 Backend Module Diagram

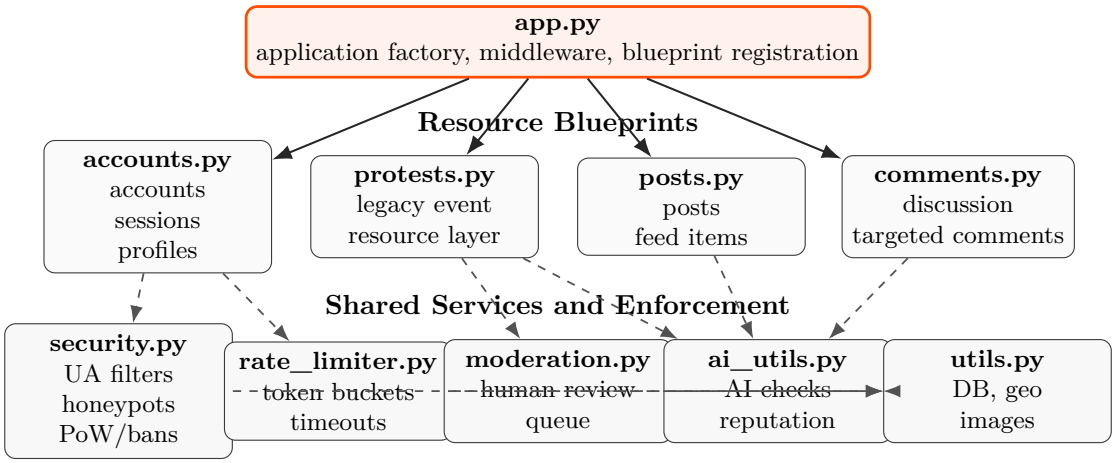

Figure 2: Backend module organization. The legacy `protests.py` module is documented as the event-resource layer in the current ShowUp product model.

3.4 Android Client Data Flow

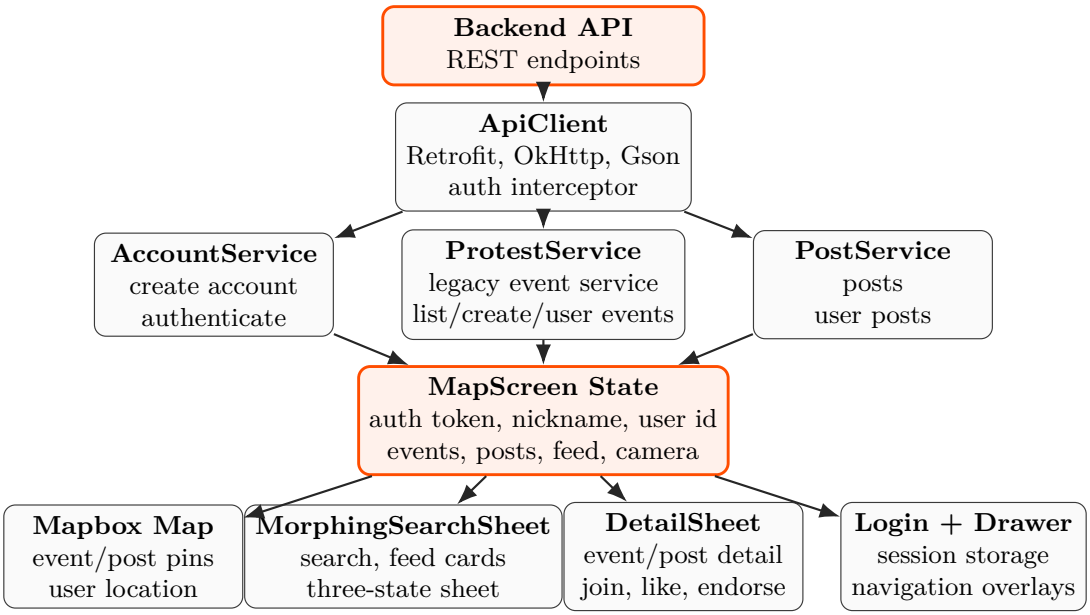

Figure 3: Android client data flow. Typed API services update central Compose state, which drives map rendering, feed presentation, detail views, authentication, and navigation.

3.5 Module Responsibilities

Module	Responsibility
`app.py`	Application entry point, environment loading, database initialization, blueprint registration, middleware registration, and response security headers.
`accounts.py`	Anonymous account creation, authentication, sessions, public profiles, nickname changes, bio updates, profile assets, and follow relationships.
`protests.py`	Event CRUD, joining/leaving, endorsements, geospatial event search, and owner-only updates/deletions. The route name is a legacy implementation namespace.
`posts.py`	Post creation, listing, geospatial filtering, user-specific posts, likes, and deletion.
`comments.py`	Comment creation, listing by target resource, and deletion.
`moderation.py`	Moderator accounts, moderator sessions, queue operations, item claiming, release, review, and queue statistics.
`ai\_utils.py`	Unified moderation pipeline, AI API requests, reputation rules, keyword extraction, event grouping, behavioral flags, and async execution.
`security.py`	User-agent filtering, honeypot routes, proof-of-work, token and fingerprint bans, HMAC request signing, and JSON validation.
`rate\_limiter.py`	Token-bucket limits, soft delays, hard timeouts, failed-auth tracking, escalation rules, and cleanup.
`utils.py`	SQLite connection management, schema initialization, haversine distance, bounding-box checks, image validation, and account cleanup.

4 Technology Stack Summary

4.1 Backend Stack

Technology	Version	Role
Python	3.11	Core backend language.
Flask	3.0.0	Web framework for routing, blueprints, and request handling.
Gunicorn	21.2.0	Production WSGI server, configured with gthread workers.
bcrypt	4.0.1	Hashing for account numbers and moderator passwords.
Pillow	10.1.0	Image validation, processing, and EXIF stripping.
OpenAI client	1.61.1	OpenAI-compatible client for moderation API calls.
requests	2.31.0	HTTP client used for IP geolocation integration.

python-dotenv	1.0.0	Environment variable loading from `.env` and overrides.
SQLite3	Built in	Five-file persistence model for accounts, events, posts, comments, and moderation.

4.2 External Backend Services

Service	Use
OpenRouter AI	Primary AI moderation provider, configured for Gemini 2.0 Flash.
Gemini / Google AI	Fallback AI API through an OpenAI-compatible endpoint.
ip-api.com	Free IP geolocation used for moderation context and geo-mismatch checks.

4.3 Android Stack

Technology	Version	Role
Kotlin	2.2.20	Android application language, targeting JVM 11.
Jetpack Compose	BOM 2024.02.00	Declarative UI framework.
Material 3	Compose Material 3	Base component system, heavily customized.
Retrofit	2.11.0	Type-safe REST API client.
OkHttp	4.12.0	HTTP engine and logging interceptor.
Gson	2.11.0	JSON serialization and deserialization.
Mapbox Maps	11.18.1	Interactive map rendering.
Mapbox Compose Extension	Matching SDK line	Compose integration for Mapbox.
Gradle	8.13	Build system.
Android Gradle Plugin	8.12.3	Android build plugin.
compileSdk / targetSdk	36	Android API target.
minSdk	29	Android 10 and newer.

4.4 DevOps and Testing

Area	Summary
Containerization	Docker using `python:3.11-slim`; Docker Compose with named volumes for database and upload persistence.
Runtime server	Gunicorn with four workers and two threads per worker.
Development connectivity	Backend base URL is configurable for local, staging, or production environments.
Backend tests	No formal backend test suite is present; `benchmark_pow.py` exists as a standalone proof-of-work benchmarking utility.
Android tests	Placeholder JUnit and Espresso tests are included.

5 Backend Deep Dive

5.1 Application Entry Point: `app.py`

The backend is created in `app.py`. Its main responsibilities are:

- Loading configuration from `.env` and `.env.override`.
- Configuring upload handling and maximum file size, with a 5 MB upload limit.
- Enabling or disabling moderation through environment configuration.
- Initializing all five SQLite databases through `utils.init_db()`.
- Registering blueprints for accounts, events, posts, comments, and moderation.
- Registering security middleware for user-agent filtering, honeypots, proof-of-work, and ban checks.
- Handling CORS preflight requests and expired account cleanup before requests.
- Applying response headers including HSTS, content security policy, `X-Frame-Options`, `X-Content-Type-Options`, and CORS headers.

5.2 Accounts Module: `accounts.py`

The accounts module implements anonymous account management. The user-visible flow is intentionally minimal: a nickname creates an account, and the returned account number becomes the authentication secret.

5.2.1 Primary Endpoints

Endpoint	Purpose
`POST /accounts/create`	Create an account from a nickname and return the secret account number.
`POST /accounts/auth`	Authenticate with an account number and receive a bearer token.
`POST /accounts/logout`	Destroy the current session.
`POST /accounts/revoke_all`	Revoke all sessions for an account.
`GET /accounts/me`	Return the authenticated user's profile.
`GET /accounts/profile/<name>`	Return a public profile by nickname.
`PUT /accounts/change_nickname`	Change nickname, subject to a three-day cooldown.
`PUT /accounts/bio`	Update profile biography.
`POST /accounts/profile_picture`	Upload a profile picture.
`POST /accounts/banner`	Upload a profile banner.
`POST /accounts/follow_user`	Follow another user.
`POST /accounts/unfollow_user`	Unfollow another user.

5.2.2 Account Number and Session Design

- Account numbers are UUID-based and generated using the `secrets` module.
- Account numbers are not stored in plaintext.
- A bcrypt hash with 12 rounds stores the account secret securely.
- A SHA-256 peppered lookup hash supports efficient account lookup without revealing the raw secret.
- Bearer tokens are generated with `secrets.token_urlsafe(32)`.
- Session lifetime is environment-driven. The documentation notes a production target of 72 hours and a testing default of 30 minutes.
- Sessions may store HMAC signing keys for optional request integrity verification.

5.3 Event Resource Module: `protests.py`

The event resource module manages the platform's primary organizing object. Its implementation uses the legacy route namespace `/protests`; product-facing behavior should be understood as event management.

5.3.1 Primary Endpoints

Endpoint	Purpose
`POST /protests/create`	Create an event resource with multipart data and optional photos.
`GET /protests`	Search and list events using geospatial and metadata filters.
`GET /protests/user/<nickname>`	Return a user's created events.
`PUT /protests/<id>`	Update an event; owner-only.
`DELETE /protests/<id>`	Delete an event; owner-only.
`POST /protests/<id>/join`	Join an event.
`POST /protests/<id>/leave`	Leave an event.
`POST /protests/<id>/endorse`	Endorse an event.
`POST /protests/<id>/unendorse`	Remove an endorsement.

5.3.2 Geospatial Search

The backend supports two geospatial search modes:

- **Center-point and radius**: Search around latitude and longitude, with a maximum radius of 150 km.
- **Bounding box**: Search by `lat_min`, `lat_max`, `lon_min`, and `lon_max`. The bounding box is validated to keep the covered area within the platform's practical search limits.

A bounding-box pre-filter is used before haversine distance calculations, improving query efficiency by eliminating obviously out-of-range rows before calculating precise distance.

5.4 Posts Module: `posts.py`

The posts module supports updates and community content attached to the broader event ecosystem.

Endpoint	Purpose
`POST /posts/create`	Create a post with optional location and photos.
`GET /posts`	Search or list posts using location, tags, limit, and offset.
`GET /posts/user/<nickname>`	Return posts created by a user.
`DELETE /posts/<id>`	Delete a post; owner-only.
`POST /posts/<id>/like`	Like a post.
`POST /posts/<id>/unlike`	Remove a like from a post.

5.5 Comments Module: `comments.py`

Comments support discussion on both event resources and posts through a shared schema. Each comment stores a `target_type` and `target_id`, allowing one comments table to attach discussion to multiple resource types.

Endpoint	Purpose
`POST /comments/create`	Create a comment against a target resource.
`GET /comments/<target_type>/<target_id>`	List comments for a post or event.
`DELETE /comments/<id>`	Delete a comment; owner-only.

5.6 Moderation Module: `moderation.py`

The moderation module is separate from regular user accounts. It manages moderator authentication, queue operations, claim locks, and final review actions.

Endpoint	Purpose
`POST /moderation/create`	Create a moderator account using a setup key.
`POST /moderation/login`	Authenticate a moderator.
`POST /moderation/logout`	End a moderator session.
`GET /moderation/me`	Return current moderator information.
`GET /moderation/queue`	List moderation queue items with pagination and status filtering.
`GET /moderation/queue/<id>`	Return a single queue item.
`POST /moderation/queue/<id>/claim`	Claim an item for exclusive review for 10 minutes.
`POST /moderation/queue/<id>/unclaim`	Release a claimed item.
`POST /moderation/queue/<id>/review`	Approve or deny an item atomically.

`GET /moderation/queue/stats`	Return pending, approved, and denied counts.

The queue lifecycle is:

```
pending and unclaimed
  -> pending and claimed
  -> approved or denied
```

Claims expire after 10 minutes of inactivity. Reviews use atomic updates constrained by `status='pending'`, preventing two moderators from reviewing the same item simultaneously.

5.7 AI Utilities: `ai_utils.py`

The AI utility layer provides the moderation engine and related support functions. Its main responsibilities are:

- `unified_moderation_engine()`: Orchestrates keyword checks, AI analysis, reputation checks, and final routing.
- `send_ai_request()`: Calls the configured AI API through an OpenAI-compatible client and uses exponential backoff on HTTP 429 responses.
- API key fallback and rotation across primary and fallback keys.
- Two-stage moderation: fast keyword filtering followed by AI analysis when needed.
- Three moderation outcomes: `ALLOWED`, `DENIED`, and `AMBIGUOUS`.
- Reputation scoring from 1 to 1000, including penalties for bot-like behavior, harmful escalation patterns, and repeated geo-mismatch signals.
- AI-assisted grouping of related events by geographic proximity.
- Keyword extraction and database update after AI analysis.
- Async execution through a thread pool to avoid blocking request handlers.

5.8 Security Middleware: `security.py`

Security is implemented through several middleware-style controls:

- User-agent blocklist for bots, scanners, crawlers, and known testing tools.
- Malformed user-agent detection for empty, null-like, or too-short values.
- Fingerprint banning using SHA-256 over IP and user-agent data.
- Permanent token banning for revoked or abusive tokens.
- Honeypot routes such as `/wp-admin`, `/.env`, `/admin`, `/config`, `/debug`, `/phpmyadmin`, and other common attack targets.
- Proof-of-work challenge-response for account creation using SHA-256 nonce search.
- Optional HMAC request signing with per-session keys.
- Strict JSON content-type validation for JSON endpoints.
- Response security headers including HSTS, CSP, `X-Frame-Options: DENY`, and `X-Content-Type-Options: nosniff`.

5.9 Rate Limiter: `rate_limiter.py`

The rate limiter uses token-bucket semantics with separate buckets for different activity types.

Bucket	Default	Purpose
`protests_per_day`	5/day	Event creation limit using the legacy bucket name.
`posts_per_day`	10/day	Post creation limit.
`reads_per_hour`	350/hour	GET request budget.
`writes_per_hour`	70/hour	POST, PUT, and DELETE request budget.

Soft limits begin at 50 percent of a bucket and apply random delays between 200 ms and 3000 ms. Hard limits at 100 percent trigger a timeout between 30 and 180 minutes. Five failed authentication attempts within 30 minutes also trigger a timeout. Repeat offenders may receive escalated proof-of-work difficulty.

5.10 Utilities: `utils.py`

The utilities module provides shared infrastructure:

- SQLite connection handling with WAL mode.
- Database schema initialization across all five database files.
- Haversine distance calculation between GPS coordinates.
- Bounding-box validation against the maximum area constraint.
- Image validation with Pillow and metadata stripping.
- File type detection through `python-magic`.
- Keyword-filter lookup operations for the moderation fast path.
- Expired account cleanup.

6 Android Frontend Deep Dive

6.1 Main Activity and `MapScreen`

The Android project follows a single-activity Compose architecture. `MapScreen()` is the root composable responsible for the map experience and most application-level state.

6.1.1 Components Managed by `MapScreen`

- `MapboxMap` composable with a custom Mapbox style.
- `MapViewportState` for camera follow, overview, and animated transitions.
- User location puck with bearing indicator.
- Dynamic GeoJSON sources and layers for event pins and post pins.
- `EdgeTiltOverlay` for map pitch gestures.

- `TiltRuler` for pitch angle display.
- `MorphingSearchSheet`, a three-state bottom sheet.
- `SheetSideButtons` pinned to the sheet edge.
- `SideMenuDrawer` for navigation and user status.
- `LoginScreen` for account creation and authentication.
- `DetailSheet` for detailed event and post views.
- `MyProtestsScreen` and `MyPostsScreen` for user-created content lists.
- `NewEventForm` for event creation.
- `CreateProtestScreen`, currently a placeholder screen.

6.1.2 State Management

- Authentication state: `authToken`, `nickname`, and `userId`.
- API state: lists of events, posts, and mixed feed items.
- UI animation state: `sheetProgress` and related derived values.
- Map state: camera center, zoom, bearing, and pitch.

6.2 Theme and Design System

`ProtestTheme.kt` implements the ShowUp design system in Compose. The class name is historical; the visual system itself is the ShowUp brand system.

Design element	Implementation
Primary color	Atomic Orange, #FF4D00. Used for CTAs, active states, pins, and urgent highlights.
Secondary color	Cool White, #E8E8E8. Used for supporting highlights and secondary text.
Neutrals	Cool gray scale from 50 to 900.
Semantic colors	Success #00C853, warning #FFB300, error #FF1744, and info #2979FF.
Typography	Space Grotesk for main UI and Space Mono for timestamps, tokens, and coordinate-like data.
Material integration	Light and dark MaterialTheme color schemes, customized around ShowUp tokens.
Dynamic tinting	Android 12+ Material You wallpaper colors can be blended into theme surfaces.

6.3 API Layer

Networking is centralized through `ApiClient`, a singleton built on Retrofit, OkHttp, and Gson.

- **Base URL**: Configurable for local, staging, or production deployments.
- **Authentication**: The bearer token is injected into requests by an OkHttp interceptor after login.

- **Timeouts**: 30 seconds for connect, read, and write operations.
- **Services**: `AccountService`, `ProtestService`, and `PostService`.

6.3.1 Data Model Files

File	Purpose
`AccountApi.kt`	Defines account request and response models, including account creation, authentication, and authentication state.
`ProtestApi.kt`	Defines event DTOs and service methods under the legacy resource name. Includes API shape and internal model transformations.
`PostApi.kt`	Defines post DTOs, post items, and post service calls.
`FeedItem.kt`	Defines a sealed class for mixed feed rows and a `mixFeed()` algorithm that interleaves events and posts at a 3:1 ratio.

6.4 UI Screens and Components

Component	Summary
`MorphingSearchSheet.kt`	Three-state draggable bottom sheet: collapsed, intermediate, and maximized. Uses spring-based snapping between states.
`SheetContent.kt`	Contains the sheet handle, search bar, tilt ruler, event cards, post cards, and action buttons.
`SheetAnimationCalculations.kt`	Computes derived animation values from `sheetProgress`, including opacity and offset transitions.
`SheetSideButtons.kt`	Provides action buttons pinned to the right edge of the sheet in intermediate and maximized states.
`LoginScreen.kt`	Full-screen two-phase account flow: create account by nickname, then authenticate with account number. Stores credentials in `SharedPreferences`.
`SideMenuDrawer.kt`	Left navigation drawer with avatar, nickname, account number preview, reputation signal meter, navigation items, share action, and hidden developer options.
`MyProtestsScreen.kt`	User event list using the historical class name. Displays status badges, dates, participant count, and endorsement count.
`MyPostsScreen.kt`	User post list showing title, preview, creation date, likes, tags, and optional location info.
`DetailSheet.kt`	Bottom sheet for both events and posts. Provides peek and expanded states, images, details, sharing, and action buttons.

`EdgeTiltOverlay.kt`	Transparent touch zones on map edges for pitch control, including haptic feedback.
`TiltRuler.kt`	Canvas-drawn horizontal ruler showing map pitch angle and transition zones.
`CreateProtestScreen.kt`	Placeholder screen with UI skeleton and coming-soon message.
`NewEventForm.kt`	Event creation form with photo attachments, description, location picker, date/time pickers, and submit state.

6.5 Authentication Flow

1. User enters a nickname.
2. The app calls `POST /accounts/create`.
3. The backend returns a secret account number.
4. User saves or copies the account number.
5. The app calls `POST /accounts/auth` with the account number.
6. The backend returns a session token and profile data.
7. Token and profile data are stored in `SharedPreferences` under `showup_auth`.
8. On app restart, the stored token is loaded and verified through `GET /accounts/me`.

7 API Reference

7.1 Base URL and Authentication

Development base URLs include `http://localhost:5000` for local backend testing, while deployed environments should use the configured backend host. Protected routes use a bearer token in the `Authorization` header.

```
Authorization: Bearer <token>
```

7.2 Accounts API

Method	Route	Auth	Response
POST	`/accounts/create`	none	account number, user id, nickname
POST	`/accounts/auth`	none	authenticated flag, token, profile data
POST	`/accounts/logout`	bearer	logout message
POST	`/accounts/revoke_all_sessions`	account number	all sessions revoked message
GET	`/accounts/me`	bearer	full user profile
GET	`/accounts/profile/<nickname>`	none	public profile

PUT	`/accounts/change_nickname`	bearer	message and new nickname
PUT	`/accounts/bio`	bearer	message and bio
POST	`/accounts/profile_picture`	bearer	uploaded profile picture filename
POST	`/accounts/banner`	bearer	uploaded banner filename
POST	`/accounts/follow_user`	bearer	updated profile
POST	`/accounts/unfollow_user`	bearer	updated profile

7.3 Events API: Legacy `/protests` Namespace

The event API currently uses the legacy `/protests` namespace because the backend was originally named around a narrower organizing concept. In the current product direction, these endpoints represent general event resources. A production-facing API could later introduce `/events` aliases while keeping `/protests` for backward compatibility during migration.

Method	Route	Auth	Response
POST	`/protests/create`	bearer	event id, tags, photos
GET	`/protests`	none	array of event resources
GET	`/protests/user/<nickname>`	none	user's event resources
PUT	`/protests/<id>`	bearer	message and event id
DELETE	`/protests/<id>`	bearer	deletion message
POST	`/protests/<id>/join`	bearer	updated profile
POST	`/protests/<id>/leave`	bearer	updated profile
POST	`/protests/<id>/endorse`	bearer	updated profile
POST	`/protests/<id>/unendorse`	bearer	updated profile

7.3.1 Event Search Query Parameters

- `lat`, `lon`: center point for radius search.
- `lat_min`, `lat_max`, `lon_min`, `lon_max`: explicit bounding box.
- `name`: partial name match.
- `date`: date match in `YYYY-MM-DD` format.
- `tags`: comma-separated tags.
- `limit`: number of results.

7.3.2 Event Creation Multipart Fields

- Required: `name`, `lat_min`, `lat_max`, `lon_min`, `lon_max`.
- Optional: `description`, `date`, `time`, and up to six photo files.

7.4 Posts API

Method	Route	Auth	Response
POST	`/posts/create`	bearer	post id, tags, photos
GET	`/posts`	none	array of posts
GET	`/posts/user/<nickname>`	none	user's posts
DELETE	`/posts/<id>`	bearer	deletion message
POST	`/posts/<id>/like`	bearer	updated profile
POST	`/posts/<id>/unlike`	bearer	updated profile

Post search parameters include location filters, tags, `limit`, and `offset`. Creation uses multipart fields: required `title` and `content`, optional location bounds, and optional photos.

7.5 Comments API

Method	Route	Auth	Response
POST	`/comments/create`	bearer	comment id and metadata
GET	`/comments/<type>/<target_id>`	none	array of comments
DELETE	`/comments/<id>`	bearer	deletion message

7.6 Moderation API

Method	Route	Auth	Response
POST	`/moderation/create`	setup key	moderator id
POST	`/moderation/login`	credentials	token and moderator id
POST	`/moderation/logout`	bearer	logout message
GET	`/moderation/me`	bearer	moderator info
GET	`/moderation/queue`	bearer	total and queue items
GET	`/moderation/queue/<id>`	bearer	single queue item
POST	`/moderation/queue/<id>/claim`	bearer	claim metadata
POST	`/moderation/queue/<id>/unclaim`	bearer	release message
POST	`/moderation/queue/<id>/review`	bearer	decision message
GET	`/moderation/queue/stats`	bearer	pending, approved, denied counts

7.7 Geographic Constraints and Static Assets

Center-point queries are limited to a 150 km radius. Bounding boxes are validated to stay within an approximately equivalent maximum span. Requests exceeding the permitted area return a `400 Bad Request`.

Static uploaded assets are served from:

- `/accounts/profile_pictures/<filename>`
- `/posts/uploads/<filename>`
- `/protests/uploads/<filename>` for event resource photos under the legacy namespace

8 Security Architecture

8.1 Defense-in-Depth Layers

Layer	Controls
Network perimeter	User-agent blocklist, honeypot routes, malformed user-agent detection, and security headers.
Rate limiting	Token buckets with soft delays, hard timeouts, separate read/write/content budgets, failed-auth tracking, and escalated proof-of-work.
Anti-bot proof-of-work	SHA-256 challenge-response on account creation with default and escalated difficulty levels.
Authentication and sessions	Anonymous account numbers, bcrypt secret storage, SHA-256 peppered lookup hashes, token-based sessions, and optional HMAC signing.
Data protection	Parameterized SQL queries, EXIF stripping, file validation through Pillow and python-magic, and 5 MB maximum uploads.
Ban system	Fingerprint-based bans, token bans, and short-lived fingerprint cache for performance.

8.2 Moderator Security

- Moderator accounts are stored separately from regular users.
- The first moderator requires `MODERATOR_SETUP_KEY`; the default value `changeme` must be replaced in production.
- Moderator sessions use a 12-hour sliding expiration model.
- Atomic claim and review operations reduce race-condition risk.
- Queue items cannot be double-reviewed because review updates are constrained to pending items.

9 AI-Powered Content Moderation

9.1 Two-Stage Pipeline

Stage	Description

Stage 1: Keyword filter	Submitted content is checked against the `keyword_filters` table. A match can trigger immediate rejection without an AI request.
Stage 2: AI analysis	Content, user reputation, IP geolocation, and context are sent to the configured AI model for structured moderation analysis.

The AI response format is structured around bracket-delimited tags:

```
[ALLOWED|DENIED|AMBIGUOUS]
[CAUSE]reason[/CAUSE]
[KEYWORDS]k1,k2[/KEYWORDS]
[GROUP]group_id[/GROUP]
[GEO_TOPIC]country[/GEO_TOPIC]
[FLAGS]flags[/FLAGS]
```

9.2 Verdicts

Verdict	Backend Behavior
`ALLOWED`	Content passes and is created immediately.
`DENIED`	Content is rejected and the API returns a reason.
`AMBIGUOUS`	Content is placed in the human moderation queue for review.

9.3 Reputation System

Reputation is scored from 1 to 1000 and influences moderation strictness.

Range	Mode	Visual Label
Below 50	Strict filtering	Untrusted
50-299	Elevated scrutiny	Low reputation
300-599	Standard filtering	Developing
600-849	Lighter monitoring	Positive
850+	Minimal intervention	Trusted

Penalties include bot-like behavior, harmful escalation signals, geo-mismatch, and repeated geo-mismatch. Rewards include participation milestones for events and like milestones for posts. Reputation is bounded between 1 and 1000.

9.4 AI API Resilience

The moderation layer uses OpenRouter as the primary API provider, with Gemini / Google Generative Language as a fallback. It supports key rotation, exponential backoff on rate limits, configurable minimum inter-request delay, and asynchronous execution through a thread pool.

9.5 Human Review Queue and Reports

Ambiguous content is stored in `moderation.db` in the `moderation_queue` table. Moderators claim items for exclusive review, release them when needed, and make final approve/deny decisions. Stale claims expire after 10 minutes.

Users can report content through `POST /moderation/report`. Reports store the reporter, content type, content id, reason, and timestamp. A takedown threshold of 10 reports can trigger automatic removal.

10 Design System: ShowUp

10.1 Brand Identity

ShowUp is positioned around presence, coordination, and collective action at scale. The logo should be described as a circular mark constructed from multiple smaller circles. Each smaller circle represents an individual participant, group, organizer, checkpoint, or local community; together they form a unified larger circle, communicating unity, turnout, shared direction, and coordinated movement without reducing the product to one event type.

The visual system is dark-first, mostly monochrome, and driven by a single charged accent: atomic orange.

10.2 Color Palette

Token	Value	Use
Primary: Atomic Orange	#FF4D00	CTAs, active states, event pins, urgent highlights.
Primary Subtle	#FF4D001A	Background tint and selected states.
Primary On	#050505	Text and icons on orange surfaces.
Secondary: Cool White	#E8E8E8	Supporting highlights and secondary text.
Secondary Subtle	#E8E8E81A	Dividers and ghost borders.
Secondary On	#050505	Text on secondary surfaces.
Success	#00C853	Positive state feedback.
Warning	#FFB300	Caution and edge-state feedback.
Error	#FF1744	Destructive or failed states.
Info	#2979FF	Informational state.

Neutral scale: #F5F5F5, #E8E8E8, #D1D1D1, #B4B4B4, #8A8A8A, #6E6E6E, #525252, #3A3A3A, #262626, and #141414.

10.3 Typography

Style	Specification	Use
Display	56 px, bold	Hero text, limited to one or two lines.
H1	40 px, bold	Page titles.
H2	28 px, semibold	Major sections.
H3	20 px, semibold	Subsections.
H4	16 px, semibold, uppercase	Fine headings.

Body Primary	16 px, regular-medium	Main reading text.
Body Secondary	14 px, regular	Supporting text.
Caption	12 px	Small explanatory text.
Label	11 px, uppercase	Controls, metadata labels, badges.
Monospace/Data	14 px	Timestamps, tokens, coordinates.

10.4 Component Philosophy

- Dark mode is the native state; light mode is a tactical inversion.
- Animations should feel mechanical and precise rather than elastic.
- Rounded interface components reinforce the circular identity system and the idea of many participants forming one organized whole.
- Elevation is encoded through lighter surfaces and increased rounding.

10.5 Key Components

Component	**Specification**
Primary button	Pill-shaped, atomic orange fill, black text, slight scale-down on press.
Secondary button	Transparent body, neutral border, white text, 12 px radius.
Ghost button	No resting border, subtle hover background, 12 px radius.
Destructive button	Pill-shaped red fill and slight press compression.
Icon button	Circular, 48 px minimum touch target.
Card	Neutral-900 background, 16 px radius, orange left border when active.
Input field	Neutral-800 background, 12 px radius, orange focus ring.
Modal/dialog	20 px radius, fade and scale animation, solid overlay.
Toast	12 px capsule, slide-in motion, 4-8 second persistence.
Badge/tag	Pill-shaped capsule, orange for live or urgent states, neutral for standard metadata.
Bottom navigation	Neutral-900 background, pill-shaped active indicator.
Skeleton loader	Neutral-800 rounded blocks with subtle shimmer.
Spinner	2 px atomic orange stroke, single-color rotation.
Progress bar	4 px pill-shaped track with atomic orange fill.

11 Database Schemas

11.1 Accounts Database: `accounts.db`

Table	**Important Fields**	**Purpose**

accounts	id, account_hash, account_number, nickname, reputation, profile fields, relationship fields	Stores user profile and account data. Account numbers are stored as hashes, not plaintext.
sessions	token, user_id, created_at, expires_at, hmac_key	Stores active user sessions.
rate_limit_buckets	user_id, bucket_type, window_start, count	Tracks usage per rate-limit bucket.
rate_limit_timeouts	id, user_id, fingerprint, reason, timeout_until	Stores soft/hard enforcement outcomes.
failed_auth_attempts	fingerprint, attempted_at	Tracks failed login attempts by fingerprint.
pow_challenges	token, challenge, difficulty, created_at	Stores proof-of-work account creation challenges.
banned_fingerprints	fingerprint, reason, created_at	Stores banned client fingerprints.
banned_tokens	token, reason, created_at	Stores banned session tokens.

11.2 Event Database: protests.db

Table	Important Fields	Purpose
protests	id, user_id, nickname, name, description, date, time, bounding coordinates, tags, photos, counts, group_id, created_at	Stores primary event resources under the legacy table name.
keyword_filters	id, keyword	Stores keyword filters used in the moderation fast path.

11.3 Posts Database: posts.db

Table	Important Fields	Purpose
posts	id, user_id, nickname, title, content, optional coordinates, tags, photos, likes_count, rep_milestones, created_at	Stores community posts and updates.

11.4 Comments Database: comments.db

Table	Important Fields	Purpose
comments	id, user_id, nickname, target_type, target_id, content, moderation_status, created_at	Stores comments attached to posts or event resources.

11.5 Moderation Database: moderation.db

Table	Important Fields	Purpose
moderators	id, username, password_hash, created_at	Stores moderator accounts.
moderator_sessions	token, moderator_id, created_at, expires_at	Stores moderator sessions.
moderation_queue	id, content_type, content_data, user_id, AI fields, status fields, claim fields, review fields, created_at	Stores ambiguous or flagged content for human review.
reports	id, reporter_id, content_type, content_id, reason, created_at	Stores user reports.

12 Deployment and Infrastructure

12.1 Docker Configuration

Item	Value
Base image	python:3.11-slim
System dependencies	build-essential, python3-dev, libmagic1
Working directory	/app
Exposed port	5000
Run command	gunicorn -bind 0.0.0.0:5000 -workers 4 -threads 2 -worker-class gthread app:app

Runtime optimizations include PYTHONDONTWRITEBYTECODE=1, PYTHONUNBUFFERED=1, and SINGLE_ACCOUNT_MODE=False for multi-account production behavior.

Uploaded images are sanitized by stripping EXIF metadata. Only genuine PNG, JPG, and GIF files are accepted, with actual file type validation rather than extension-only checks.

12.2 Docker Compose

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - data:/app/data
      - uploads:/app/uploads
    env_file: .env
    environment:
      - DB_DIR=/app/data
      - UPLOAD_FOLDER=/app/uploads
```

Named volumes preserve SQLite databases and uploaded images across container restarts.

12.3 Runtime Configuration

`configure.py` allows runtime environment changes without rebuilding the image. It writes updates to `.env.override` and sends `SIGHUP` to Gunicorn for graceful reload.

12.4 Android Build Configuration

Item	Value
Module	`:app`
Package ID	`com.example.myapplication`
Version	`1.0`, `versionCode 1`
Compile SDK	36
Minimum SDK	29, Android 10 and newer
Target SDK	36
Build types	Debug and release, with minification disabled
Mapbox repository	Mapbox Downloads Maven repository requiring a Mapbox account token
Gradle wrapper	8.13
Kotlin plugin	2.2.20 with Compose compiler plugin

Required permissions are `INTERNET`, `VIBRATE`, `ACCESS_COARSE_LOCATION`, and `ACCESS_FINE_LOCATION`.

12.5 Development Setup

12.5.1 Backend

1. Install Python 3.11.
2. Install dependencies with `pip install -r requirements.txt`.
3. Create a `.env` file with required AI API keys and security configuration.
4. Start the Flask development server with `python app.py`.

12.5.2 Android

1. Set `sdk.dir` in `local.properties`.
2. Set the Mapbox download token in `~/.gradle/gradle.properties`.
3. Configure the backend base URL in the API client or through in-app developer options.
4. Build with `./gradlew assembleDebug`.
5. Install the APK on an Android 10+ device or emulator.

12.5.3 Frontend-to-Backend Connectivity

The Android app communicates with the backend through a configurable base URL. Local development can target a local or LAN-accessible backend, while staging and production builds should point to a stable hosted API endpoint. A hidden developer options dialog in the side menu can override the base URL at runtime for testing.

13 Environment Variables and Configuration

13.1 Core Backend Variables

Variable	Purpose or Default
`AI_API_URL`	Default: `https://openrouter.ai/api/v1`.
`AI_API_KEY`	Primary API key for moderation.
`AI_FALLBACK_KEY_1`	First fallback API key.
`AI_FALLBACK_KEY_2`	Second fallback API key.
`AI_MODEL`	Default moderation model: `google/gemini-2.0-flash-001`.
`ENABLE_MODERATION`	Enables or disables content moderation.
`LOOKUP_PEPPER`	Cryptographic pepper for account lookup hashing.
`SINGLE_ACCOUNT_MODE`	Development option; production should allow multiple accounts.

13.2 Moderation Configuration

Variable	Purpose or Default
`MODERATION_API_BASE_URL`	Base URL for moderation AI calls.
`MODERATION_MODEL`	Model used for moderation.
`MODERATION_MIN_REQUEST_INTERVAL_MS`	Minimum interval between AI requests; default 200 ms.
`MODERATION_QUEUE_WARN_THRESHOLD_MS`	Queue wait warning threshold; default 2000 ms.
`MODERATION_GROUPING_RADIUS_KM`	Radius for grouping related events; default 10 km.

MODERATOR_SETUP_KEY	Initial moderator setup secret; default changeme must be replaced.

13.3 Rate Limiting and Security Configuration

Variable	Purpose or Default
RATELIMIT_PROTESTS_PER_DAY	Event creation bucket, legacy variable name; default 5.
RATELIMIT_POSTS_PER_DAY	Post creation bucket; default 10.
RATELIMIT_READS_PER_HOUR	Read request budget; default 350.
RATELIMIT_WRITES_PER_HOUR	Write request budget; default 70.
RATELIMIT_FAILED_AUTH	Failed authentication threshold; default 5.
RATELIMIT_FAILED_AUTH_WINDOW_MIN	Failed authentication window; default 30 minutes.
HARDLIMIT_HASH_DIFFICULTY	Escalated proof-of-work difficulty; default 8.
POW_DIFFICULTY	Default proof-of-work difficulty; default 6.
SESSION_TTL_MINUTES	Session token lifetime; testing default noted as 30 minutes.
ENABLE_REQUEST_SIGNING	Enables optional HMAC request signing; default false.

13.4 Android Configuration

Configuration	Description
Mapbox access token	Stored in res/values/mapbox_access_token.xml.
Mapbox style URL	mapbox://styles/trydud/cmnvuk89c002t01sa7nx161tz.
Backend base URL	Hardcoded during development or overridden through developer options.
SharedPreferences key	showup_auth, storing token, nickname, and account number.

14 Project File Inventory

14.1 Backend Repository

File or Directory	Purpose
app.py	Flask application entry point.
accounts.py	Account, authentication, and profile management.
protests.py	Event resource CRUD and geospatial search under legacy naming.
posts.py	Post CRUD and geospatial search.
comments.py	Comment handling.

moderation.py	Moderation queue and moderator review workflow.
ai_utils.py	AI moderation engine.
utils.py	Database, geospatial, and image helper functions.
security.py	Security middleware.
rate_limiter.py	Token-bucket rate limiter.
configure.py	Runtime configuration tool.
benchmark_pow.py	Proof-of-work benchmarking utility.
requirements.txt	Python dependency list.
Dockerfile	Container build instructions.
docker-compose.yml	Multi-container orchestration.
API_DOCUMENTATION.md	Full API reference.
MODERATION_DASHBOARD_INTEGRATION.md	Moderator dashboard guide.
FRONTEND_INTEGRATION_GUIDE.md	API guide for frontend integration.
curls.md	Ready-to-use curl command examples.
documentation.txt	Quick reference and cheat sheet.
accounts.db, protests.db, posts.db, comments.db, moderation.db	SQLite database files.
uploads/	Uploaded image directory.
openrouter_docs/	Local OpenRouter API documentation files.

14.2 Android Repository

File or Directory	Purpose
MainActivity.kt	Entry point and MapScreen orchestration.
ProtestTheme.kt	ShowUp theme, colors, and typography under historical file naming.
AccountApi.kt	Authentication API data models.
ProtestApi.kt	Event resource API client under legacy naming.
PostApi.kt	Post API data models.
FeedItem.kt	Feed types and mixing algorithm.
LoginScreen.kt	Account creation and login flow.
SideMenuDrawer.kt	Navigation drawer.
MorphingSearchSheet.kt	Main bottom sheet interaction model.
SheetContent.kt	Sheet content and feed cards.
SheetAnimationCalculations.kt	Animation math for sheet transitions.
SheetSideButtons.kt	Side action buttons.
MyProtestsScreen.kt	User event list under legacy naming.
MyPostsScreen.kt	User post list.
CreateProtestScreen.kt	Placeholder event-creation screen under legacy naming.
NewEventForm.kt	Event creation form.
DetailSheet.kt	Detail view for events and posts.

`EdgeTiltOverlay.kt`	Map tilt gesture zones.
`TiltRuler.kt`	Map pitch ruler.
`app/build.gradle.kts`	App-level build configuration.
`build.gradle.kts`	Root Gradle build configuration.
`settings.gradle.kts`	Project module settings.
`gradle/libs.versions.toml`	Version catalog.
`showup-design-system/`	Brand guidelines, colors, typography, components, imagery, voice, style guide, and tokens.
`hazedocs/`	Haze Compose blur library documentation.
`fonts/`	Font assets for the brand system.

The Android project contains roughly 5,800 lines of Kotlin across 19 source files.

14.3 Agent Skill Directories

Both repositories include AI agent skill definitions for development context. They cover areas such as cybersecurity analysis, Docker expertise, Flask API development, Android architecture, Android design guidelines, Mapbox Android patterns, brand strategy, and custom design-system guidance.

Conclusion

ShowUp is best presented as a privacy-conscious event coordination platform with strong map-based discovery, anonymous authentication, reputation-aware moderation, and layered anti-abuse protections. Its main engineering merit is not one isolated feature, but the integration of backend security, AI moderation, human review, geospatial search, and a polished Android experience into one coherent system.

For examination purposes, the most important framing is that ShowUp has evolved into a generalized large-scale event organizing platform. Historical protest-related implementation names should be explained as code-level legacy terms inherited from an earlier product direction, while the product itself should be described through its current scope: organizing, discovering, and moderating public events at scale.